PYTHON REVISION SHEET

Utkarsh - Day 1 Restart (Feb 16, 2026)

COMMON MISTAKES TO AVOID

1. Indentation Errors

Variables initialized inside loops reset every iteration. Put them BEFORE the loop.

❌ Wrong:

```python
while(n>0):
    count = 0  # resets every time!
```

✅ Right:

```python
count = 0
while(n>0):
```

2. Return Inside Loop

Putting return inside a loop exits after first iteration.

❌ Wrong:

```python
for i in range(n):
    return i  # exits immediately
```

✅ Right: Complete the loop, THEN return

3. = vs ==

- `=` is assignment
- `==` is comparison

❌ `if(n%2 = 0)`
✅ `if(n%2 == 0)`

4. && doesn't exist

Python uses `and` and `or`, not `&&` and `||`

5. Destroying variables before comparing

If you modify n in a loop, save original first: `org = n`

6. Forgetting to assign

`sum//10` does nothing. Need `sum = sum//10` or `sum //= 10`

KEY PATTERNS

Extract digits from right:

```python
last_digit = n % 10
remove_last = n // 10
```

Build number from digits:

```python
rev = rev * 10 + digit
```

Loop until found (while True):

```python
while True:
    if(condition):
        return answer
    increment += 1
```

Simplify True/False returns:

❌ Wrong:

```python
```

```
if(a==b):
    return True
else:
    return False
```

✅ Right:

```python
return a == b
```

PROBLEMS COMPLETED

Problem	Key Logic
Sum of Digits	`sum += n%10, n//=10`
Count Digits	`count += 1, n//=10`
Reverse Number	`rev = rev*10 + n%10`
Palindrome	`reverse == original`
GCD	from min down, first that divides both
LCM	from max up, first both divide into
Factorial	`fact *= i` for i in 1 to n
Power	`result *= base`, y times
Armstrong	sum of (digit^count) == original
Perfect Number	sum of divisors (excl self) == n
Strong Number	sum of factorial(digit) == n
Neon Number	sum of digits of n^2 == n
Automorphic	n^2 % (10^digits) == n
Harshad	n % sum_of_digits == 0
Disarium	sum of digit^position == n

YOUR PATTERNS (BE HONEST)

Stalling tactics you use:

- Asking "how many more questions?"
- Saying "this is boring"
- Requesting summaries/context switches
- Worrying about time complexity before basic logic works
- Starting new chats instead of coding

What actually works:

- Writing code before asking questions
- Running code before pasting
- Getting it working first, optimizing later
- Library > room for focus

NEXT: STRINGS

Basics to know:

- `s[0]` = first char
- `s[-1]` = last char
- `len(s)` = length
- Strings are IMMUTABLE - can't do `s[0] = 'x'`
- Build new strings instead: `new_s = new_s + char`

First string problem: Reverse a string

```
reverse_string("hello") → "olleh"
```

No using `[::-1]`

CONTEXT FOR NEW CHAT

Copy this to new chat if limit hits:

```
CONTEXT FOR CONTINUATION:

Utkarsh - Day 1 restart after week gap (food poisoning post-GATE).

Completed revision: sum_of_digits, reverse_number, palindrome, GCD, LCM, factorial, power, armstrong - all from memory.

Current task: Starting STRINGS. First problem: reverse_string("hello") → "olleh" without [::-1]

His patterns: stalls by asking questions, says "boring", makes big promises he breaks. Push hard, demand code, no coddling.

Deadline: ₹50k/month job by April 2026.

Your tone: "Paste the code." "Did you run it?" "Stop talking. Code."
```

Deadline: ₹50k/month job by April 2026

Stop planning. Start coding.